Componente TOTP

Generación de QR y Validación de Códigos Temporales

Fecha: 29/04/2026 | Máster en Ciberseguridad

Alberto Galindo Llorente

1. Introducción

Este documento describe el diseño e implementación de un componente software que permite generar un código QR para registrar un servicio TOTP en aplicaciones como Google Authenticator, y posteriormente validar los códigos de 6 dígitos que dicha aplicación genera cada 30 segundos.

TOTP (Time-based One-Time Password) es el estándar definido en el RFC 6238, construido sobre HOTP (RFC 4226). Utiliza HMAC-SHA1 aplicado sobre el par (secreto compartido, contador de tiempo), donde el contador equivale al número de intervalos de 30 segundos transcurridos desde la época UNIX. El resultado se trunca a 6 dígitos decimales.

2. Tecnologías utilizadas

Plataforma: Aplicación web (Python 3.12)

Framework: Flask 3.x – micro-framework WSGI ligero

TOTP: pyotp 2.9 – implementación RFC 6238 / RFC 4226

Código QR: qrcode 7.x + Pillow – generación de imágenes PNG

Frontend: HTML5 / CSS3 / Vanilla JS (sin dependencias externas)

Documentación: python-docx – generación de este documento

3. Arquitectura y estructura de ficheros

El proyecto se organiza en los siguientes ficheros:

- app.py → Servidor Flask con las rutas y la lógica TOTP
- templates/index.html → Interfaz web (QR + formulario de validación)
- requirements.txt → Dependencias Python del proyecto
- documentacion.pdf → Documento descriptivo en PDF

4. Funcionamiento del componente

4.1 Generación del secreto y el código QR

Al acceder por primera vez a la ruta '/', Flask genera un secreto aleatorio de 160 bits en Base32 mediante pyotp.random_base32(). Este secreto se almacena en la sesión del usuario (cookie firmada) y se usa para construir la URI de aprovisionamiento otpauth://, que incluye el nombre del servicio, el emisor y el propio secreto. La librería qrcode convierte dicha URI en una imagen PNG que se embebe en el HTML como cadena Base64.

```
secret = pyotp.random_base32()          # 160 bits aleatorios en Base32
totp   = pyotp.TOTP(secret)
uri    = totp.provisioning_uri(
             name='TareaSeguridad',
             issuer_name='MasterCiberseguridad')
# Resultado: otpauth://totp/TareaSeguridad?secret=XXXX&issuer=...
qr = qrcode.make(uri)   # genera imagen PNG
```

4.2 Validación del código TOTP

El endpoint POST /validate recibe el código de 6 dígitos introducido por el usuario. Se instancia pyotp.TOTP con el secreto de sesión y se invoca totp.verify(code, valid_window=1), que acepta el código del intervalo actual y del inmediatamente anterior/posterior (tolerancia de ±30 s por deriva de reloj).

```
totp  = pyotp.TOTP(session['totp_secret'])
valid = totp.verify(code, valid_window=1)
# True  -> Código correcto
# False -> Código incorrecto o expirado
```

5. Rutas de la API REST

- GET / → Página principal: genera secreto, muestra QR y formulario
- POST /validate → Recibe campo 'code', devuelve JSON {valid: bool, message: str}
- POST /regenerate → Genera nuevo secreto y recarga la página (nuevo QR)

6. Instrucciones de instalación y ejecución

Requisitos previos: Python 3.10 o superior instalado en el sistema.

```
# 1. Instalar dependencias
pip install -r requirements.txt

# 2. Arrancar el servidor de desarrollo
python app.py

# 3. Abrir el navegador en:
http://127.0.0.1:5000
```

Una vez abierta la web, escanear el código QR con Google Authenticator (Android o iOS). La aplicación mostrará un código renovable de 6 dígitos cada 30 segundos. Introducir dicho código en el campo de validación y pulsar 'Verificar código' para comprobar si es correcto.

7. Consideraciones de seguridad

- El secreto TOTP nunca viaja en texto plano; se embebe en el QR o la sesión cifrada.
- En producción, almacenar el secreto en BD con cifrado en reposo (AES-256).
- La clave de sesión Flask debe ser un valor fuerte: secrets.token_hex(32).
- Añadir rate-limiting al endpoint /validate para mitigar ataques de fuerza bruta.
- valid_window=1 permite tolerancia de ±30 s; valores mayores reducen la seguridad.
- HTTPS es obligatorio en producción para proteger la cookie de sesión.

8. Diagrama de flujo simplificado

```
[Usuario] --GET /----> [Flask genera secreto] --> [pyotp crea URI otpauth://]
                                    |
                       [qrcode genera PNG en Base64]
                                    |
               <-- HTML con QR + formulario --------

[Usuario escanea QR con Google Authenticator]
[Introduce código de 6 dígitos en el formulario]

[Usuario] --POST /validate--> [Flask] --> [pyotp.TOTP.verify()]
                                                    |
                           <-- JSON {valid, message} --
```

9. Referencias

- RFC 6238 – TOTP: Time-Based One-Time Password Algorithm
- RFC 4226 – HOTP: An HMAC-Based One-Time Password Algorithm
- PyOTP library – https://github.com/pyauth/pyotp
- qrcode library – https://github.com/lincolnloop/python-qrcode
- Flask documentation – https://flask.palletsprojects.com

- Google Authenticator (Android) – https://play.google.com/store/apps/details?id=com.google.android.apps.authenticator2
- Google Authenticator (iOS) – https://itunes.apple.com/es/app/google-authenticator/id388497605